

LoadTestingTool User Manual

Scope: Current runner and desktop UI, including CaseQL, brace-only assertions, dynamic values, correlation, environments, unexpected-field handling, Excel editing, comparison reports, and troubleshooting.

1. Overview

LoadTestingTool executes API, GraphQL, database, file, and C# script test steps defined in an Excel workbook. It supports sequential and threaded execution, multiple environments, multiple data rows, iterations, response assertions, variables extracted from earlier steps, result workbooks, event logs, and a desktop UI for managing test instances.

The workbook is the executable test definition. The runner loads it, resolves templates and dynamic values, runs each enabled step, evaluates assertions, writes result files, and returns a non-zero exit code when a testcase fails.

The current assertion syntax uses braces. Use `{eq}`, `{contains}`, `{exists}`, and similar prefixes. The old `eq:value` and `contains:path` syntax is not supported by the current runner.

2. Distribution layout

A typical installation looks like this:

```
LoadTestingTool-distribution/
├── LoadTestingTool.UI.exe
├── instances/
│   └── DataEngine_ähs3Nytt/
│       ├── DataEngine.xlsx
│       ├── Templates/
│       ├── Results/
│       ├── History/
│       └── Logs/
└── appsettings.json
```

Location	Purpose
Templates	GraphQL documents, HTTP bodies, scripts, and other external templates.
Results/pass	Completed runs with no failure.
Results/failed	Completed failed runs and comparison workbooks.
Results/.running	Temporary staging while a run is active.
History	History output when configured.
Logs	UI and general logs. Run-specific logs are stored with results.

Excel creates temporary lock files named like `~$DataEngine.xlsx`. These are not valid workbooks. The UI ignores them and the runner rejects them if passed directly. The runner opens the real workbook with shared read-only access, so it can read the last saved version while Excel has the workbook open. Unsaved Excel edits are not visible until Excel saves the workbook, and the runner never writes to `DataEngine.xlsx` during execution.

3. Workbook requirements

The workbook must contain these worksheets:

```
config
request
response
```

Worksheet names are matched case-insensitively.

3.1 config sheet

Each row defines one step. Required identifying columns are `testcaseindex`, `testcase`, and `stepname`.

Column	Description
<code>testcaseindex</code>	Numeric testcase identifier.
<code>testcase</code>	Testcase name.
<code>stepname</code>	Unique name within the testcase. It prefixes variables and assertion paths.
<code>steptype</code>	<code>HTTP</code> , <code>GraphQL</code> , <code>CaseQL</code> , <code>SQL</code> , <code>File</code> , or <code>Script</code> . Default: <code>HTTP</code> .
<code>templetsource</code>	Relative HTTP body template. Required for HTTP.
<code>targeturl</code>	Target URL. Required for HTTP, GraphQL, and CaseQL. Supports <code>{env}</code> .
<code>stepconfig</code>	JSON configuration for non-HTTP steps.
<code>stage</code>	<code>Setup</code> , <code>Test</code> , or <code>Teardown</code> . Default: <code>Test</code> .
<code>enabled</code>	<code>true</code> or <code>false</code> .
<code>timeout</code>	Step timeout in seconds.
<code>verb</code>	HTTP method, normally <code>GET</code> or <code>POST</code> .
<code>contenttype</code>	Request content type.
<code>threads</code>	Worker count for threaded execution.
<code>iterations</code>	Number of iterations.
<code>threadintervalms</code>	Delay between worker starts.
<code>iterationintervalms</code>	Delay between iterations.
<code>wait</code>	Delay before the step.
<code>expectedstatus</code>	Expected HTTP status expression.
<code>stoponfailure</code>	Stops the current sequence after failure. Default: <code>true</code> .
<code>assert</code> or <code>assertenabled</code>	Enables assertions. Default: <code>true</code> .

Column	Description
<code>assertonlyresponse</code>	Checks only response paths explicitly configured in the response sheet.
<code>ignoreempty</code>	Ignores unexpected null, empty-string, <code>{}</code> , and <code>[]</code> values.
<code>environment</code> or <code>environments</code>	Comma- or pipe-separated environments.

A testcase-index and step-name pair must be unique.

3.2 HTTP steps

HTTP steps load their request body from `templetsource` . The file is resolved under the configured Templates folder. Variables are substituted into URLs, headers, and bodies.

```
StepType: HTTP
Verb: POST
TargetUrl: https://api-{env}.example.test/orders
TempletSource: create-order.json
```

Header columns use the `header_` prefix, for example `header_Authorization` and `header_Content-Type` .

3.3 GraphQL steps

GraphQL steps use JSON in `stepconfig` :

```
{
  "Source": "caseql1.graphql",
  "OperationName": "getCase",
  "Variables": {
    "caseId": "_caseId_"
  }
}
```

Use **Inline** instead of **Source** when the query is stored in the workbook configuration. GraphQL output is published under the **data** runtime variable.

3.4 CaseQL steps

CaseQL is GraphQL with built-in form flattening. It converts this:

```
{
  "data": {
    "cases": [{
      "forms": [{
        "fields": [{"name": "registreringsnummer", "value": "ABC123"}]
      }]
    }]
  }
}
```

into this:

```
{
  "data": {
    "cases": [{
      "forms": {"registreringsnummer": "ABC123"}
    }]
  }
}
```

The assertion path is therefore:

```
data.cases[0].forms.registreringsnummer
```

A separate flattening script is not needed.

3.5 SQL steps

SQL steps use a connection profile and command settings:

```
{
  "ConnectionProfile": "MainDatabase",
  "CommandType": "Text",
  "CommandText": "select Id, Status from Orders where ExternalId = @externalId",
  "Parameters": {"externalId": "_externalId_"},
  "Outputs": {"orderId": "Id", "orderStatus": "Status"}
}
```

Connection profiles are configured in `appsettings.json`. SQL output mappings become runtime variables.

3.6 File steps

File steps can read or write files:

```
{
  "Action": "Read",
  "Path": "payloads/input.json",
  "OutputVariable": "fileContent",
  "JsonPath": "data.id"
}
```

Supported options include `Action`, `Path`, `Content`, `OutputVariable`, and `JsonPath`.

3.7 Script steps

Scripts can be loaded from a file or supplied inline:

```
{"Source": "transform.csx"}
```

or:

```
{"Inline": "Runtime.Set(\"result\", Runtime.Get(\"data\")); return Runtime.Get(\"result\")"}
```

Scripts can use `Runtime.Get` , `Runtime.Set` , `Runtime.Assert` , and `Runtime.Log` . Trusted scripts are disabled by default and are subject to `ScriptTimeoutSeconds` .

4. Request data

The `request` sheet requires `testcaseindex` , `testcase` , and `dataid` .

Variables use the form:

```
<stepname>.<variable>
```

Example:

TestcaseIndex	Testcase	DataId	caseql1.caseId	createOrder.customer
1	Case API	MM2	12345	C001

The optional `dontrun` request column contains comma-separated step names to skip for that data row on every iteration. For example:

TestcaseIndex	Testcase	DataId	dontrun	caseql1.caseId
1	Case API	MM2	caseql2,createOrder	12345

The `caseql2` and `createOrder` steps are skipped for `MM2` . Names are matched case-insensitively. Leave `dontrun` blank to run all steps.

The `caseql1.caseId` value is available to `caseql1` as `_caseId_` . A step variable is referenced in templates and configuration using underscores:

```
_caseId_  
_customer_
```

Each `dataid` must be unique for a testcase.

5. Response assertions

The `response` sheet requires `testcaseindex`, `testcase`, and `dataid`. Assertion columns use:

```
[optional brace verb]stepname.response.path
```

Examples:

```
{eq}caseql1.data.cases[0].status  
{contains}caseql1.data.cases[0].message  
{notempty}caseql1.data.cases[0].id  
{exists}caseql1.data.cases[0].id
```

The expected value is placed in the data cell below the header.

A plain cell value uses the header verb:

```
Header: {contains}caseql1.data.cases[0].status  
Cell:   Approved
```

A brace verb in the cell overrides the header:

```
Header: {contains}caseql1.data.cases[0].status  
Cell:   {eq}Approved
```

The old `eq:value`, `contains:value`, `eq:path`, and `contains:path` forms are not supported by the current runner.

5.1 Assertion verbs

Verb	Meaning	Example
<code>eq</code>	Equal, case-insensitive.	<code>{eq}Approved</code>
<code>ne</code>	Not equal.	<code>{ne}Rejected</code>
<code>lt</code>	Numeric less-than.	<code>{lt}100</code>
<code>lte</code>	Numeric less-than-or-equal.	<code>{lte}100</code>
<code>gt</code>	Numeric greater-than.	<code>{gt}100</code>
<code>gte</code>	Numeric greater-than-or-equal.	<code>{gte}100</code>
<code>contains</code>	Contains text.	<code>{contains}Approved</code>
<code>notcontains</code>	Does not contain text.	<code>{notcontains}Error</code>
<code>startswith</code>	Starts with text.	<code>{startswith}ABC</code>
<code>endswith</code>	Ends with text.	<code>{endswith}123</code>
<code>regex</code>	Matches a regular expression.	<code>{regex}^[A-Z]{3}[0-9]+\$</code>
<code>exists</code>	Path exists.	<code>{exists}</code>
<code>notexists</code>	Path does not exist.	<code>{notexists}</code>
<code>empty</code>	Value is empty or null.	<code>{empty}</code>
<code>notempty</code>	Value is not empty.	<code>{notempty}</code>
<code>in</code>	Value is one of pipe-separated values.	<code>{in}Open Closed</code>
<code>size</code>	String length equals the expected number.	<code>{size}6</code>

For `exists` , `notexists` , `empty` , and `notempty` , the cell usually contains only the brace verb.

5.2 Optional missing fields

A missing response path is ignored when the expected cell is blank and the effective operator is equality. Explicit assertions remain strict. For example, `{exists}`, `{notempty}`, and `{eq}required-value` still fail when the path is missing.

5.3 Unexpected fields

By default, a non-empty response path that is not configured in the response sheet is reported as an unexpected contract failure.

Use:

```
AssertOnlyResponse: true
```

to validate only configured paths.

Use:

```
IgnoreEmpty: true
```

to ignore unexpected null, empty-string, `{}`, and `[]` values. Non-empty unexpected values remain failures unless `AssertOnlyResponse` is enabled.

6. Dynamic values

Dynamic values are compiled when the workbook is loaded. Values from the same source cell are cached, so a request and its response reference use the same generated value.

6.1 Date and time

```
<datum>  
<currentdatum>  
<currenttime>  
<currentdatetime>  
<currenttimestamp>
```

Custom formats use .NET date and time format strings [1](#):

```
<datum:yyyyMMdd>  
<currenttime:HHmmss>  
<currentdatetime:yyyyMMdd_HHmmss_fff>  
<currenttimestamp:yyyyMMddHHmmssfff>
```

Use `fff` for milliseconds.

6.2 Random numbers

```
<randomnumber>  
<randomnumber:6>  
<randomnumber:000000>  
<randomnumber:1000-9999>
```

The first form uses the default generator. The second requests six digits. The third requests six zero-padded digits. The fourth selects an inclusive range.

The legacy form `<randomnumber_6>` remains supported by the current runner.

6.3 GUIDs

```
<guid>  
<guid:D>  
<guid:N>  
<guid:B>  
<guid:P>
```

`D` is the standard hyphenated form. `N` has no hyphens. `B` uses braces and `P` uses parentheses. `<guid:N>` is usually safest for API fields.

6.4 Workbook references

Use this syntax to reference a source cell:

```
<From_request_MM.Registreringsnummer_2>
```

The source cell is compiled first. If it contains `<randomnumber:6>`, `<guid:N>`, or a date token, the compiled value is used by the assertion. Workbook compilation caches the result so the request and assertion compare the same value.

7. Correlation and extraction

The response sheet can contain an `extractvariable` column. Mappings use:

```
stepname.response.path:variableName
```

Multiple mappings are separated by `|`:

```
caseql1.data.cases[0].id:caseId|caseql1.data.cases[0].token:authToken
```

Later steps use the extracted values as:

```
_caseId_  
_authToken_
```

Correlation is maintained within the current data row and execution context. Threaded workers do not share mutable correlation values with each other.

8. Environments and execution modes

URLs can contain `{env}`:

```
https://api-{env}.example.test/graphql
```

The selected environment is also available as the runtime variable `env`.

Supported execution modes are:

Mode	Behavior
<code>threaded</code>	Runs workers concurrently.
<code>loop</code>	Sequential loop execution.
<code>sequential</code>	Alias for sequential execution.

Thread and iteration behavior is controlled with `threads`, `iterations`, `threadintervals`, `iterationintervals`, and `wait`. `stoponfailure=true` stops the current sequence after a failed step.

9. Command-line runner

The runner accepts:

Option	Purpose
<code>--excel</code> or <code>--workbook</code>	Workbook path.
<code>--templates</code>	Templates folder.
<code>--results</code>	Results folder.
<code>--internal-log</code>	<code>true</code> or <code>false</code> .
<code>--instance-id</code>	Instance identifier.
<code>--run-id</code>	Explicit run identifier.
<code>--environments</code>	Comma-separated environment selection.
<code>--execution-mode</code>	<code>threaded</code> , <code>loop</code> , or <code>sequential</code> .
<code>--dataids</code>	Data-ID selection expression.
<code>--testcases</code>	Testcase selection expression.

Example:

```
LoadTestingTool.exe ^  
  --excel "C:\LoadTestingTool\instances\DataEngine_ähs3Nytt\DataEngine.xlsx"  
  --templates "C:\LoadTestingTool\instances\DataEngine_ähs3Nytt\Templates" ^  
  --results "C:\LoadTestingTool\instances\DataEngine_ähs3Nytt\Results" ^  
  --environments "U,T,PT" ^  
  --execution-mode sequential ^  
  --testcases 1
```

Data-ID selection uses testcase groups:

```
1:MM1,MM2;2:PT1
```

Testcase `0` normally means all testcases. Exit code `0` means all executed testcases passed. A non-zero code indicates a failed testcase or startup/preflight failure.

10. Desktop UI

The UI discovers instance folders, ignores `~$` Excel lock files, launches the runner, monitors NDJSON events, and displays live instance and assertion information.

10.1 Running an instance

Select an instance, choose testcase, data-ID, environment, and execution-mode settings, then run it. The UI passes these values to the runner. It also passes diagnostic `uiLaunchId` and `uiLaunchAt` values so launch delays can be compared with runner timestamps.

10.2 Assertion panel

The assertion list is refreshed manually to avoid rebuilding assertion controls on every event poll. The summary can update while assertion details remain stable.

The panel supports:

- filtering by environment;
- selecting failed assertions;
- clearing selections;
- refreshing assertion details;
- applying selected values;

- creating and opening comparison workbooks.

10.3 Apply selected value

For an existing assertion, the UI writes the actual value into its existing response-sheet cell.

For an unexpected field, the UI adds a response header using:

```
StepName.ResponsePath
```

It writes the actual payload value into the matching data row and highlights the new header and value cell green.

Before saving, the UI creates:

```
DataEngine.xlsx.backup.YYYYMMDD_HH:mm:ss.xlsx
```

It skips `<missing>` values and skips cells containing any angle-bracket token, including:

```
<From_request_...>  
<randomnumber:6>  
<guid:N>  
<datum:yyyyMMdd>
```

This prevents measured values from replacing dynamic expressions.

10.4 Comparison workbooks

The UI can create a workbook under the failed-results folder with these sheets:

- `Comparison` : grouped assertion values by environment;
- `FailedOnly` : groups containing failures;
- `Differences` : groups whose result or actual value differs;
- `Summary` : assertion, pass, fail, missing, and pass-rate totals.

The comparison workbook is separate from `DataEngine.xlsx` .

11. Results and logs

A run is first written to `Results/.running` and then moved to `Results/pass` or `Results/failed`.

Typical files are:

```
log/internal.log
log/process.log
events_<timestamp>_<testcase>_<guid>.ndjson
dataid_<selection>.xlsx
```

Important log messages include `REQUEST SENT`, `RESPONSE RECEIVED`, `ASSERTIONS COMPLETED`, `ASSERTION FAILED`, `STEP TEMPLATE LOAD`, and `REQUEST COMPLETED`.

For UI performance diagnosis, compare UI diagnostic timestamps with `runnerStartedAt`, request durations, assertion events, and event-file timestamps.

12. Configuration

The main configuration file supports these settings:

Setting	Purpose
<code>ExcelFilePath</code>	Default workbook.
<code>TemplatesFolder</code>	Default Templates folder.
<code>LogsFolder</code>	General log folder.
<code>HistoryFolder</code>	History folder.
<code>ResultsFolder</code>	Result root.
<code>RequestTimeoutSeconds</code>	Default request timeout.
<code>RunScenariosInParallel</code>	Parallel testcase/environment scheduling.
<code>TestcaseSelection</code>	Default testcase expression.
<code>PromptForTestcaseSelection</code>	Prompts at startup when enabled.
<code>InternalLoggingEnabled</code>	Enables internal logs.
<code>SaveRequestBodyToHistory</code>	Stores request bodies in history.
<code>SaveResponseBodyToHistory</code>	Stores response bodies in history.
<code>MaskSensitiveData</code>	Masks configured sensitive data.
<code>WorkspaceRoot</code>	Root used to restrict file access.
<code>AllowTrustedScripts</code>	Enables trusted scripts.
<code>ScriptTimeoutSeconds</code>	Script timeout.
<code>Connections</code>	SQL connection profiles.

Command-line options override corresponding configuration values.

13. Troubleshooting

Lock-file error

If a log shows `~$DataEngine.xlsx`, Excel was open or the UI selected its temporary file. Close Excel and use the real `DataEngine.xlsx`.

Missing assertion

Check the exact step name, path, and zero-based array index. For CaseQL, use `forms.fieldName`, not `forms[0].fields[0].value` after flattening.

Unexpected field

Add it through **Apply selected value**, set `AssertOnlyResponse=true`, or use `IgnoreEmpty=true` when only empty extra values should be ignored.

Reference mismatch

Check that the reference points to the correct `request` sheet header and row:

```
<From_request_MM.Registreringsnummer_2>
```

The source cell is compiled before the assertion reference is resolved.

Script failure

Check trusted-script configuration, workspace restrictions, source paths, runtime variable names, and script timeout.

UI lag

Refresh the assertion list manually. Compare the UI log, `internal.log`, `process.log`, and NDJSON event timestamps to identify whether the delay is in UI polling, process launch, the target service, or the runner.

14. Recommended workflow

Close Excel before running or modifying a workbook. Start with one testcase in sequential mode. Confirm that templates load, URLs resolve, dynamic values are correct, correlations are present, and assertions use the correct paths. Then increase iterations, environments, and threads gradually.

Use CaseQL for form payloads. Use brace verbs in headers and cells. Keep required fields explicit with `{exists}`, `{notempty}`, or a non-blank expected value. Use `AssertOnlyResponse=true` when an API legitimately returns many fields outside the tested subset.

Before applying assertion values, review the selected failures. Dynamic-variable cells are intentionally protected. Keep the generated workbook backup until the updated test has passed.

15. Quick examples

CaseQL step

```
StepType: CaseQL  
Verb: POST  
TargetUrl: https://api-{env}.example.test/graphql  
StepConfig: { "Source": "caseql1.graphql", "OperationName": "getCase" }
```

Required assertion

Header:

```
{notEmpty}caseql1.data.cases[0].id
```

Cell:

```
{notEmpty}
```

Dynamic equality assertion

Header:

```
{eq}caseql1.data.cases[0].id
```

Cell:

```
<From_request_MM.CaseId_2>
```

Generated request values

```
<guid:N>  
<randomnumber:6>  
<datum:yyyyMMdd_HHmss_fff>
```

16. Glossary

Assertion: A rule comparing a response value with an expected condition.

CaseQL: GraphQL execution with built-in form-field flattening.

Correlation: Publishing a value from one step for use by a later step.

Data ID: The unique identifier of one request-data row.

Environment: A target deployment such as `U`, `T`, or `PT`.

Response path: A dot-and-index path such as `data.cases[0].id`.

Threaded mode: Concurrent worker execution.

Sequential mode: One execution sequence at a time.

References

Document owner: Manus AI

Last updated: 2026-09-25

Source map: `WorkbookReader.cs`, `AssertionEngine.cs`, `TestRunner.cs`, `GraphQLAndFileExecutors.cs`, `SqlAndScriptExecutors.cs`, `Program.cs`, and `MainWindow.cs` contain the corresponding implementation areas.

Before distributing a workbook, verify the three required sheets, unique data IDs, valid brace verbs, existing template files, correct response paths, and a real workbook filename that does not begin with `~$`.